

10 Chapter 10: Sequential Circuit Design

10.1 Introduction

To review modeling of gate delay, we consider intrinsic factors, extrinsic factors, and operating conditions when estimating effective gate delay.

In addition, gates will often be *asymmetric*, meaning that the delay associated with different transitions will be different, and we may represent this effect using a timing arc table.

Gates are normally not used in isolation, but chained together, so the challenge is to calculate total delay through a complex circuit. One method of doing this is to simplify the circuit and its delays to single numbers, and then finding the *critical path*. This involves evaluating every path from input to output, and finding the worst case – this worst case is the critical path. Obviously, the result is an oversimplification, and the critical path can be easily calculated using software tools.

Critical path analysis can be simplified or complicated as needed, by including/excluding variables such as asymmetric rise/fall delays.

In practice, we sample nodes in a CMOS circuit to determine whether they are high or low (1 or 0). The space between a 1 and a 0 is known as the "eye", and clock synchronization determines the sampling rate and accuracy of sampling. However, there are similar delay concerns regarding the clock that there are with any other CMOS circuits. The functionality of any sequential circuit is highly dependent on the delay considerations.

Sequential circuits require "memory" components, like flip-flops and latches, to separate past, current, and future inputs. The clock frequency determines the speed at which this "pipeline" flows.

Sequential circuits generally include clocks, combinational logic elements, and registers, with multiple clocking methodologies developed over the years. Getting the clock edge to every part of the circuit at the same time is not easy – generally, an input clock will fan out to a number of repeaters, and these fan out to twice as many repeaters, etc, until all registers are covered; wire lengths, though, should be consistent, so an H-tree layout is a common approach.

A clock can be defined as follows in a CMOS circuit, based on its properties: the clock will have a period and frequency that is based on threshold voltages and rise/fall times. All sequential circuits have maximum clock frequencies, above which they are not guaranteed to operate. Minimum clock frequency may be as low as zero, depending on whether the circuit has functionality that can be met without a clock signal; however, dynamic sequential circuits have minimum frequencies due to the natural discharging of capacitors used as memory (DRAM).

Latches are level sensitive, where flip-flops (the components of registers) are edge triggered. Timing diagrams specify the operation of sequential circuits and their timing elements as a function of the clock period, and may include the associated propagation and contamination delays.

We will not cover latch and flip-flop design from scratch.

10.3 Static Sequencing Element Methodology

I think this might be Chapter 11 content

The operation of a latch is represented using a timing diagram. Open/closed (opaque) state. Propagation delays are important factors in D-latch operation (as with any gate), notated as $t_{pLH(GQ)}$, etc. Timing parameters for a D-latch include the setup and hold time (**what does setup and hold mean?**), as well as the minimum pulse width of the high G state. D input should target the overlap between the opening and closing of G. Datasheet example of a D-latch, gives parameters we just talked about. Some setup and hold

values are negative, meaning that setup rising high can occur *before* the clock opening the gate, and the data will still be captured.

We can put two latches back to back, with output and input buffers, to form a D flip-flop. Explanation of how this kind of dynamic flip-flop works. In this kind of back-to-back flip-flop, the data goes through two "layers" of flip-flop to ensure no data is lost? (**check this functionality**). This is an **edge-triggered** flip-flop.

D flip-flops have a specific timing diagram and parameters, similar to a latch. Setup and hold, in this case, defines the window for capturing data, as usual (either setup or hold can be negative, but their sum must be greater than zero). This allows us to derive max delay for a flip flop as

$$t_{pd} \leq T_c - (t_{\text{setup}} + t_{pcq}) \text{ **check the derivation of this.**}$$

Sum of the delays around the flip flop loop (with feedback/buffer) is a function of the clock frequency, the setup time, and the Elmore delay, so that gives us the minimum clock frequency (**I think?**)

Max clock frequency is limited by the critical (longest) path in the device. Quiz will include min/max clock frequencies dependent on critical paths through a circuit.

Enable is a common addition to a flip-flop that turns it into a feedback loop; you may also gate the clock to enable/disable the clock. Many flip-flops don't need to change that often in, say, a register file, so we can save power by disabling the clock when not needed. Reset is another common feature that sets output low, usually used for starting up and getting a known state, can be synchronous or asynchronous.

There are also min/max delay requirements/specs for flip-flops, has to do with hold time violations.

Looks at slides/example of hold timing path. When is hold constraint met/not met? More importantly, how can we fix this? Sequencing methods. Max delay derivation **check on this too.**

What is time borrowing?

Clocks are not ideal, unfortunately; this leads to clock skew! The clock arrives at different times in different places, unfortunately, and a clock also has a rise and fall time, that is likely asymmetric. Clock jitter, also bad! Caused by noise. Statistical analysis of jitter?

10.4 Metastability & Synchronization

Back to D flip-flops. In summary, at the rising edge of the clock, the D-input is held in a feedback loop and output to the Q-output, until the next rising edge of the clock. The D input must hold a *stable* 0 or 1 during the rising edge of the clock, defined by the setup and hold times. There is max time t_{CQ} in which the Q output must take on the value of the D input to operate correctly, by definition.

In synchronization, it becomes more difficult to determine the sequence of two events the closer they occur to each other in time. Closer the race, the harder to call. When the order of events is not possible to resolve, the synchronizer will enter a **metastable** state, and may remain there indefinitely. While this is happening, the output may not be correct.

A simple example would consist of a NAND arbiter (or S-R latch). This is a way of *arbitrating* between two possible values. In one case, let's say A and B arrive at the same time (the difference is less than the gate delay). Due to the physics of the device, this generates a voltage difference ΔV , which is then amplified (William Dally textbook). If the voltage eventually reaches V_{DD} , the output will stabilize over a period known as the decision time, t_d . If $\Delta t = 0$, t_d grows to infinity! In practice, this does not occur, because even small amounts of electromagnetic noise will cause the circuit to stabilize in one state or another.

This concept extends to latches and flip-flops as well, which can become similarly trapped in a metastable

state for an indeterminate amount of time. When do things go metastable, how do we deal with it?